

INF2007 – TN4 – Melissa Moya

1. Approche

Le programme accepte un paramètre `--type` pour choisir entre entiers et flottants et génère un tableau de 1 000 000 d'éléments avec `rand.NewSource(42)` pour la reproductibilité. Deux fonctions spécialisées, `computeSineSumInt` et `computeSineSumFloat`, isolent la boucle de calcul par type afin que les benchmarks mesurent uniquement le coût de `math.Sin`.

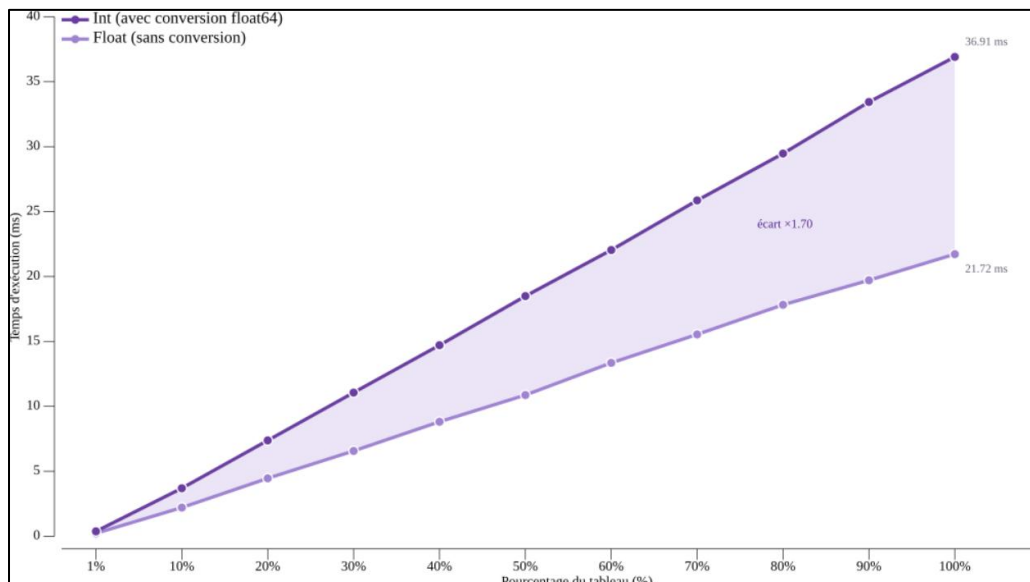
2. Résultats des benchmarks

Les 22 sous-benchmarks (11 paliers $\times$ 2 types) ont été exécutés avec `go test -bench=. -benchmem -count=6` sur un Intel i5-10300H à 2,50 GHz (Windows/amd64) et analysés avec `benchstat`. Les tableaux sont pré-générés hors de `b.N`, `b.ResetTimer()` exclut le setup, et `b.ReportAllocs()` confirme 0 B/op et 0 allocs/op.

Tableau 1 – Temps de calcul par type et pourcentage du tableau

% du tableau	Éléments	Int (ms)	Float (ms)	Ratio
1 %	10 000	0.373	0.223	1.67×
10 %	100 000	3.69	2.21	1.67×
20 %	200 000	7.38	4.46	1.66×
30 %	300 000	11.06	6.57	1.68×
40 %	400 000	14.71	8.81	1.67×
50 %	500 000	18.49	10.87	1.70×
60 %	600 000	22.04	13.35	1.65×
70 %	700 000	25.86	15.54	1.66×
80 %	800 000	29.47	17.82	1.65×
90 %	900 000	33.44	19.71	1.70×
100 %	1 000 000	36.91	21.72	1.70×

Graphique 1 – Temps de calcul selon le pourcentage du tableau (Int vs Float)



3. Analyse

Les flottants sont 1,65 à 1,70× plus rapides. L'écart ne vient pas de la précision numérique, qui reste équivalente puisque *math.Sin* opère toujours sur des *float64*, mais de la conversion *float64(v)* à chaque itération pour les entiers. Cette conversion génère l'instruction CPU *CVTSD* (Convert Scalar Integer to Scalar Double), absente du chemin Float. S'y ajoute le coût de réduction d'argument de *math.Sin* pour les grands entiers. *sin(500)* nécessite environ 79 réductions modulo 2π , alors que les flottants dans $[0, 1)$ sont déjà dans le domaine principal. De 50 % à 100 %, le temps double presque exactement pour les entiers (18,49 à 36,91 ms), confirmant la complexité $O(n)$.

La hiérarchie mémoire de l'i5-10300H explique le comportement aux grands paliers. Avec 256 KB de L1, 1 MB de L2 et 8 MB de L3, les petits paliers (1 à 10 %, 80 à 800 KB) tiennent en L2. À partir de 30 % (2,4 MB), le L3 prend le relais. À 90 % (7,2 MB), le tableau approche la limite du L3 ; la progression reste régulière jusqu'à 100 %, mais la pression croissante sur la bande passante L3 aux paliers élevés contribue aux intervalles de confiance plus larges observés à ces paliers.

go build -gcflags=-m confirme que *math.Sin* est inliné dans *sinesum.go*. La boucle *sum += math.Sin(v)* crée une dépendance de données sur l'accumulateur, ce qui empêche le pipeline FPU de superposer les additions, comme dans l'exemple *prefixSum* du chapitre 6. Les appels *math.Sin(v)* bénéficient de l'exécution dans le désordre mais sont plafonnés par la latence intrinsèque de *math.Sin*. Utiliser plusieurs accumulateurs fusionnés en fin de calcul réduirait la dépendance de données et constituerait la principale piste d'optimisation.

4. Applications numériques

Les médianes benchstat à 100 % donnent 36,9 ns par sinus pour les entiers et 21,7 ns pour les flottants.

Q1 — Distance parcourue par la lumière pendant un sinus ($c = 299\,792\,458$ m/s)

Type	Temps	Distance
Int	36,9 ns	$299,792,458 \times \frac{36,9}{10^9} \approx 11,1$ m
Float	21,7 ns	$299,792,458 \times \frac{21,7}{10^9} \approx 6,5$ m

Q2 — Nombre de sinus par tick à 120 fps (tick = 8 333 333 ns)

Type	Temps/sinus	Sinus/tick
Int	36,9 ns	$8,333,333 \div 36,9 \approx 226\,000$
Float	21,7 ns	$8,333,333 \div 21,7 \approx 384\,000$